

Flexbox

Lesson 15



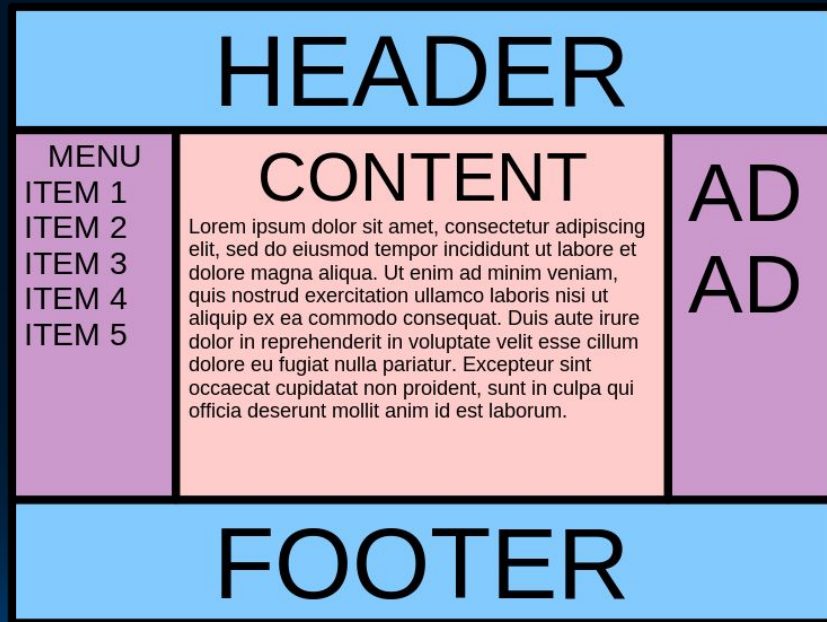


flexbox

Flexbox allows control over:

- Size
- Order
- Alignment of elements across multiple axes
- Distribution of free space between elements
- and more

- ★ Ease in creating complex layouts
- ★ Automatic alignment
- ★ Responsiveness



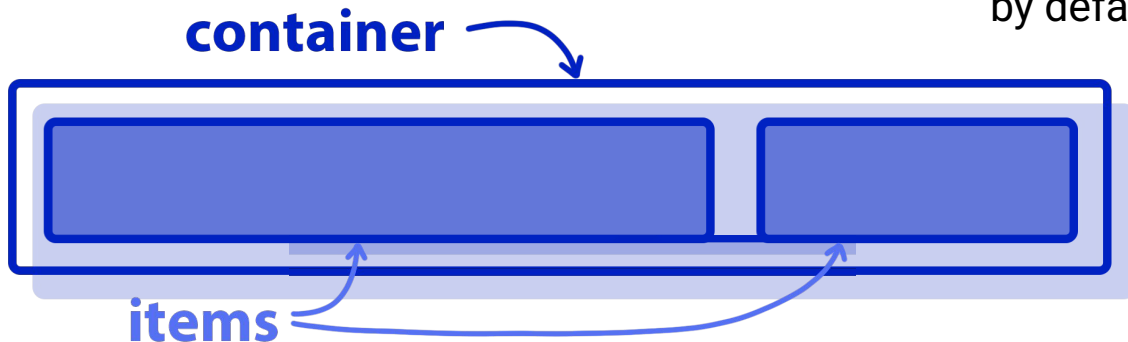
flex container and items

The container is given the property:

```
display: flex;
```

Style properties are applied to the **elements** inside the container.

The elements adjust to the width of their content and are arranged in a row by default.



flex: axis

There are two axes:
main axis ~~horizontal~~
cross axis ~~vertical~~

The direction of the axes can be changed using the property:

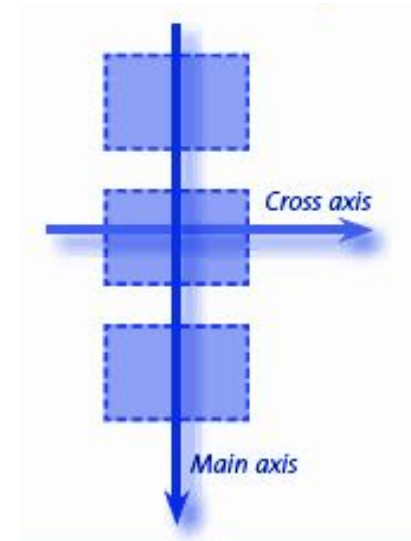
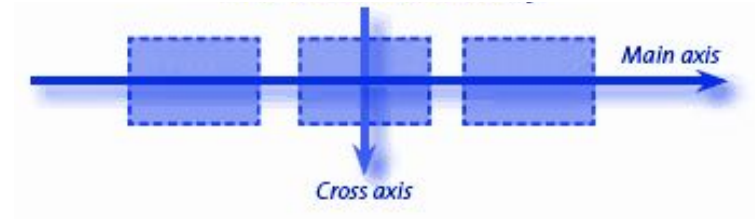
flex-direction

Default:

flex-direction: row;

To arrange elements in a column:

flex-direction: column;

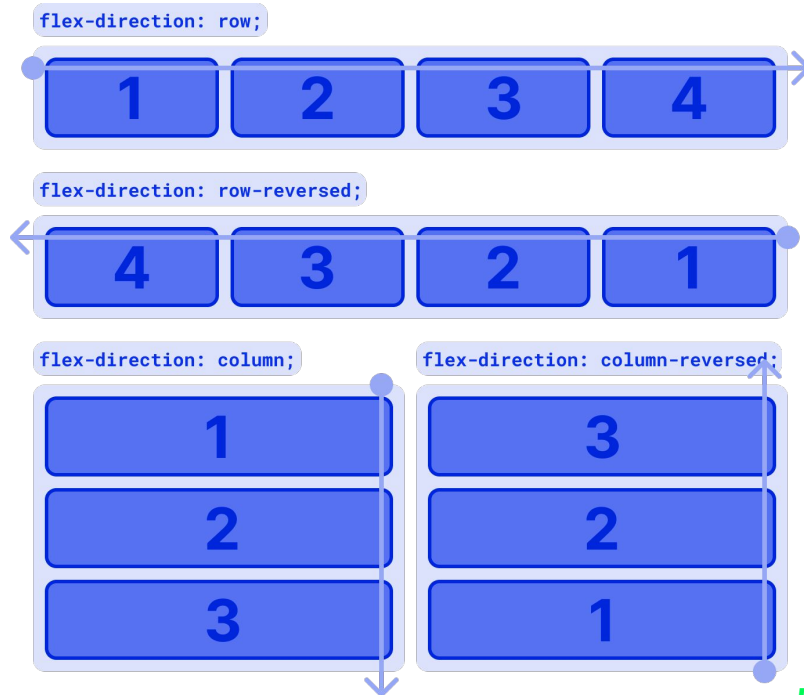


flex: Changing the Order of Elements

Demo

1. Using Reverse Direction:

```
flex-direction: row-reversed;  
flex-direction: column-reversed;
```



flex: Changing the Order of Elements

Demo

2. The **order** Property:

order: 4;

Accepts positive and negative values.

Works correctly when the **order** property is set for all elements within the flex container.



flex: Alignment elements

Main Axis:

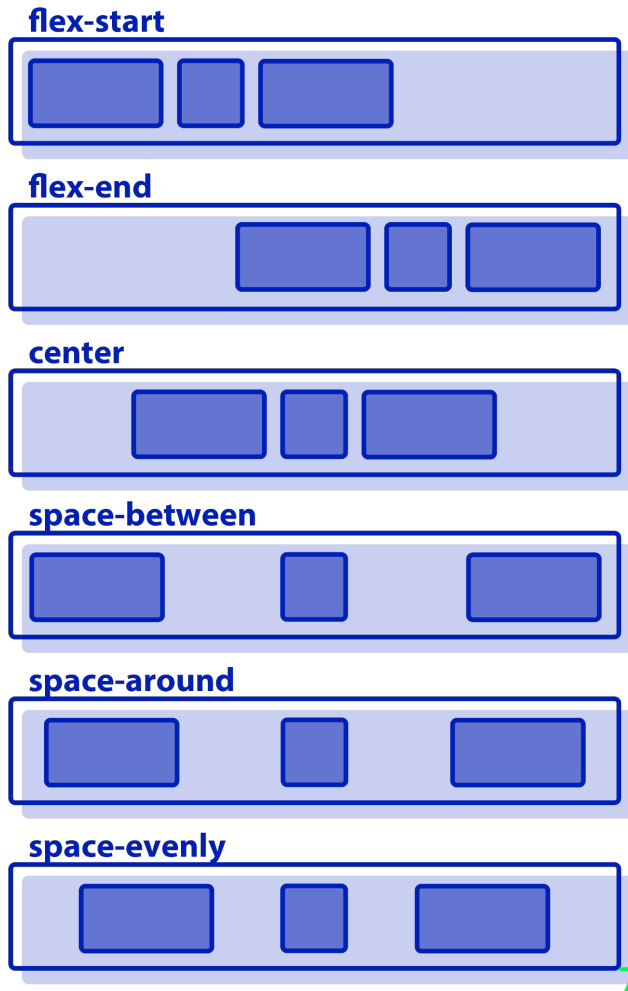
justify-content 

Default Value:

flex-start 

Demo

These property is set on the container, and the alignment is applied to the items within it.



flex: Alignment elements

Cross Axis:

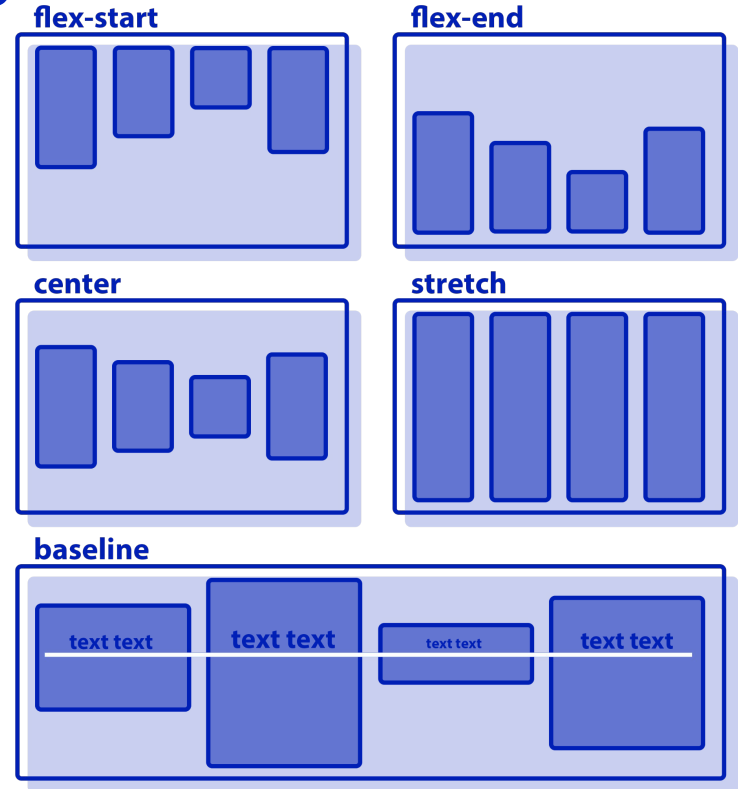
align-items 

Default Value:

stretch 

Demo

These property is set on the container, and the alignment is applied to the items within it.

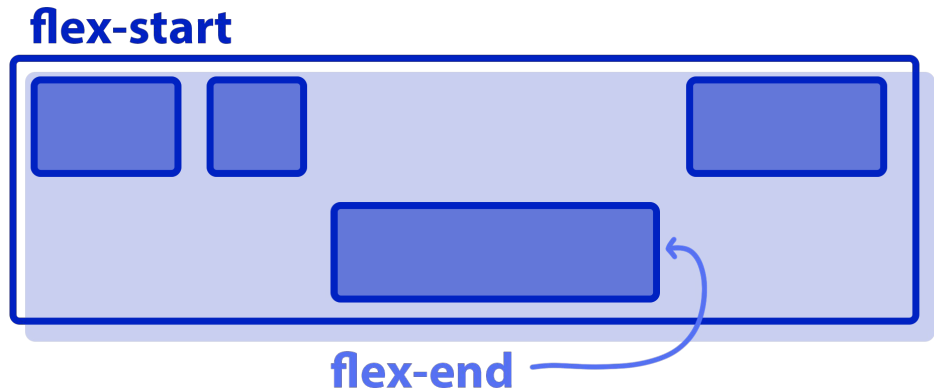


flex: Alignment one element

You can align a specific element only along the cross axis using:

align-self 

The values are the same as those for **align-items**



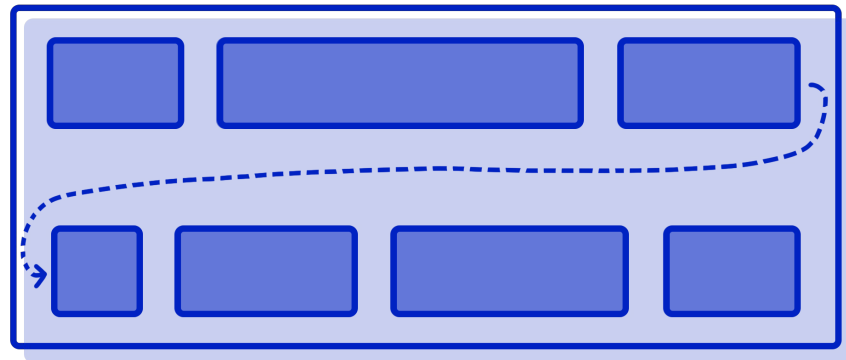
flex: Working with Rows

Flexbox can wrap items to new rows using :

```
flex-wrap: wrap;
```

Default:

```
flex-wrap: no-wrap;
```



flex: Working with Rows

Aligning Rows on the Cross Axis:

align-content

Default:

align-content: flex-start;



flex: Spacing Between Items

You can set **margin** for each element individually, but it's much easier to set spacing for the entire container in one place using the **flex** container.

gap – Sets spacing between items

gap : 10px 15px;

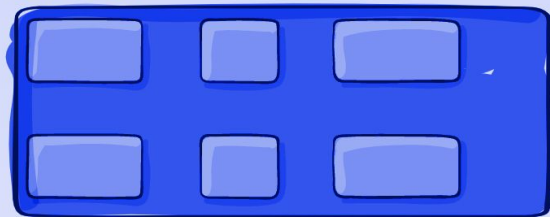
row-gap : 15px;

column-gap : 10px;

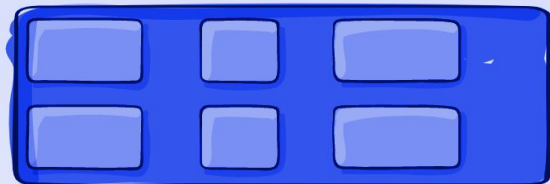
gap: 10px



gap: 30px



gap: 10px 30px



flex: distributing extra space

Applied along the **main axis** for **flex-item**.

Coefficient for distributing extra space:

flex-grow 

Values are positive integers greater than 0.

flex-grow: 2; 

The distribution is calculated based on a specific formula ->

Demo

flex-grow calculation

$$\text{new width} = \left(\frac{\text{flex grow}}{\text{total flex grow}} \times \text{free space} \right) + \text{width}$$

width: 

flex-grow: 

calculation: $\left(\frac{1}{4} \times 400 \right) + 100$ $\left(\frac{0}{4} \times 400 \right) + 100$ $\left(\frac{3}{4} \times 400 \right) + 100$

new width: 

 samanthaming  samanthaming.com  samantha_ming

flex: shrinking elements

Applied along the **main axis**.

Shrink coefficient when items don't fit in one line:
flex-shrink

Values are positive integers greater than 0.

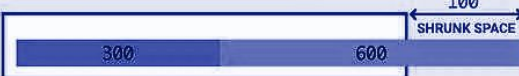
flex-shrink: 1;

The shrinkage is calculated based on a specific formula →

flex-shrink: 0; – The element will maintain the width specified by **width** or **flex-basis**, even if this causes other elements to overflow.

flex-shrink calculation

new width = width – (shrunk space × shrink ratio)

width: 

flex-shrink:

calculation:

total shrink scaled width = $\sum (\text{width} \times \text{flex shrink})$
 $(300 \times 4) + (600 \times 6) = 4800$

shrink ratio = $(\text{width} \times \text{flex shrink}) / \text{total shrink scaled width}$
 $(300 \times 4) / 4800 = 0.25$ $(600 \times 6) / 4800 = 0.75$

new width = width – (shrunk space × shrink ratio)
 $300 - (100 \times 0.25) = 275$ $600 - (100 \times 0.75) = 525$

new width: 

@ samanthaming samanthaming.com samantha_ming

flex: base size of an element

flex-basis defines the size of an element along the **main axis** before considering **flex-grow** and **flex-shrink**.

Values can be in **px** or **%**.

flex-basis: 50%; 

Child	flex-basis
flex-basis: auto	flex-basis: auto
flex-basis: 50%	flex-basis: 50%
flex-basis: auto	flex-basis: 500px

flex-basis vs widths	
<pre>... .child { ✓ flex-basis: 500px width: 100px }</pre>	flex-basis wins 500px
<pre>... .child { ✓ min-width: 100px flex-basis: 500px }</pre>	min-width wins 100px
<pre>... .child { ✓ max-width: 700px flex-basis: 500px }</pre>	max-width wins 700px

flex: shorthand property

flex-grow / flex-shrink / flex-basis

```
flex: 1 0 300px;
```

Flex

1	1	auto px %
---	---	-----------

Grow

Shrink

Basis

Centering Content, Full-Width Background

The background of the website should always stretch across the full width of the page.

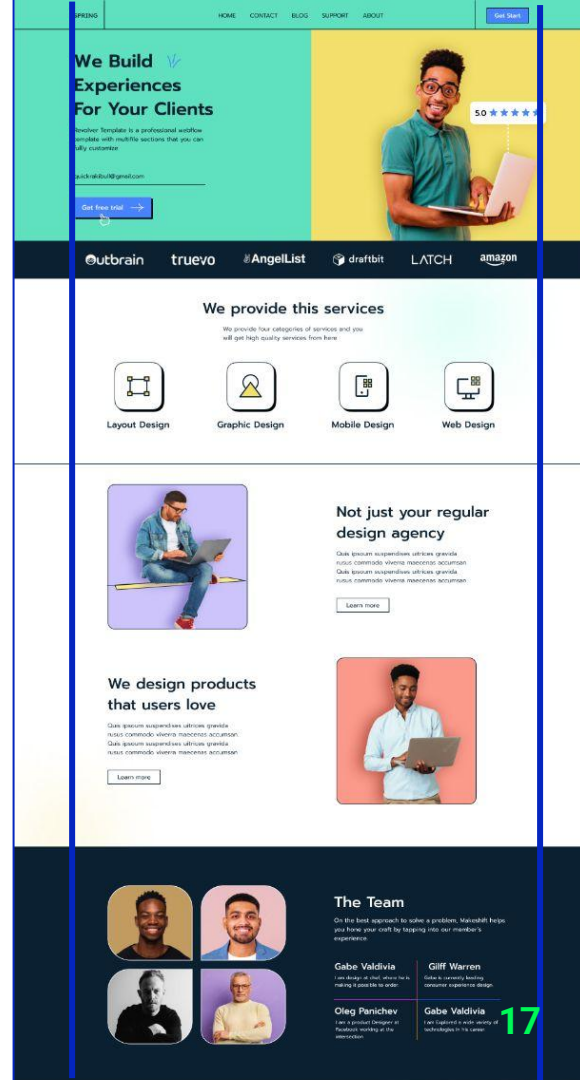
The content has a width restriction (set by width/max-width) as per the design layout and is centered on the page.

```
<section>
  <div class="content-wrapper">

section { background-color: grey; }

.content-wrapper {
  width: 1400px;
  margin: 0 auto;
}
```

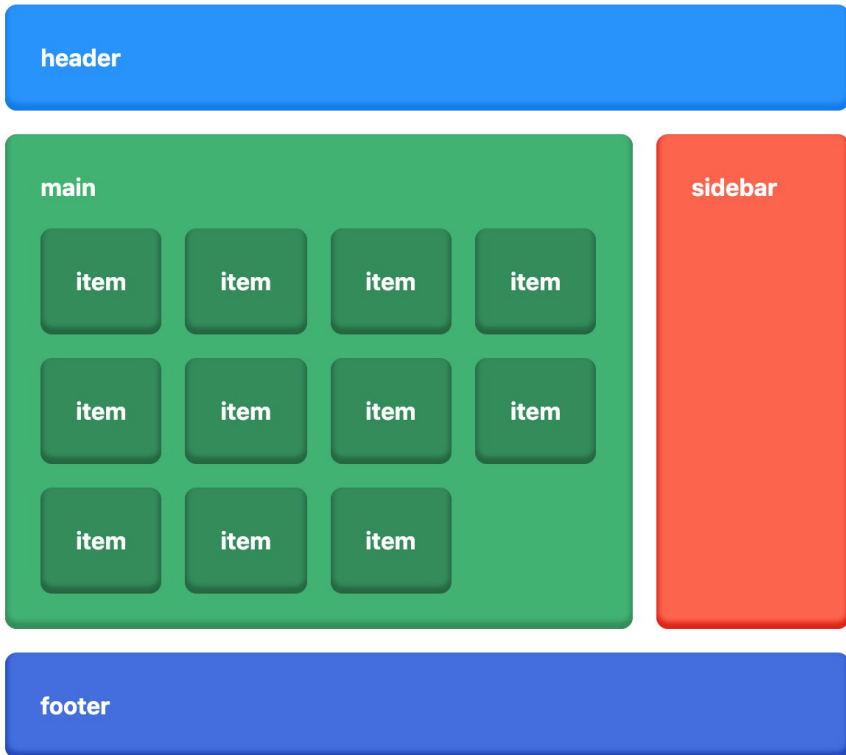
Demo



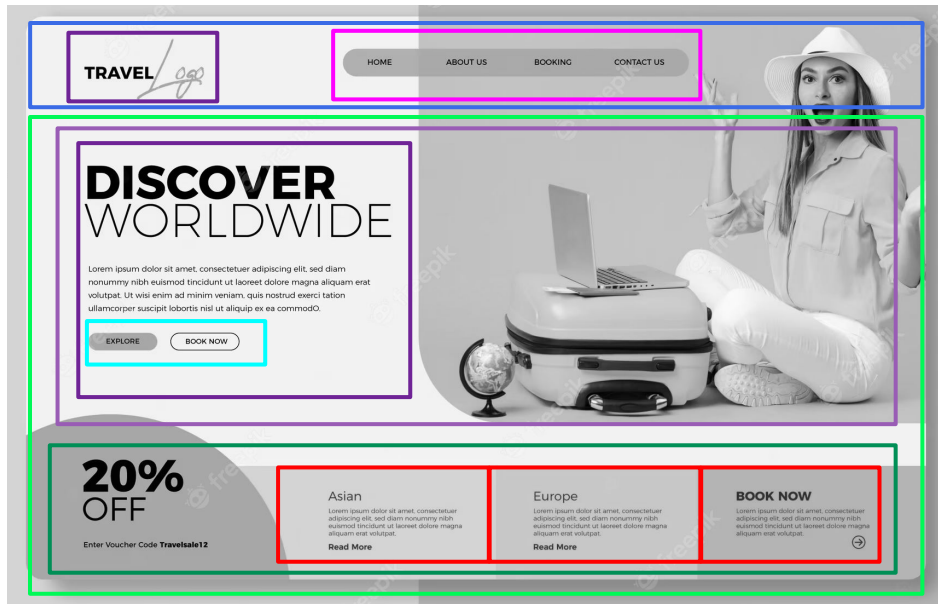
example

flex golden layout

```
body {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 20px;  
}  
  
header,  
footer {  
  flex-basis: 100%;  
}  
  
main {  
  flex-basis: 80%;  
  display: flex;  
  flex-wrap: wrap;  
}  
  
item {  
  flex-grow: 1;  
}  
  
sidebar {  
  flex-basis: 20%;  
}
```

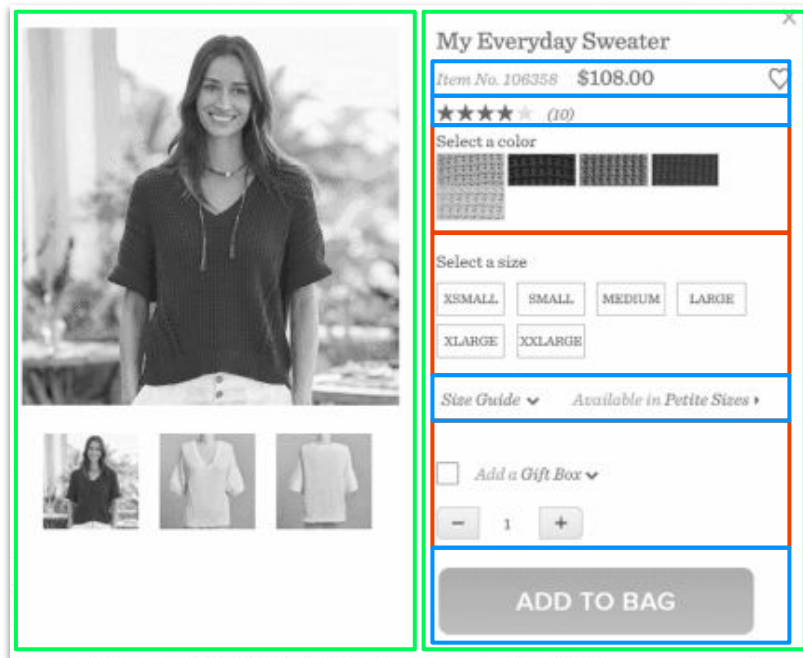


flex hero section



```
.header {  
    display: flex;  
}  
.nav {  
    display: flex;  
    margin: 0 auto;  
}  
.main {  
    display: flex;  
    flex-direction: column;  
}  
.hero-text {  
    display: flex;  
    flex-direction: column;  
    margin-right: auto;  
}  
.button-wrap {  
    display: flex;  
}  
.advantages {  
    display: flex;  
}  
.advantages-item {  
    flex-grow: 1;  
}
```

flex card item



`.is-gallery`

`.is-info`

`justify-content: space-between`

`justify-content: flex-start`

`justify-content: space-between`

```
.card {  
  display: flex;  
}  
.part {  
  flex-grow: 1;  
  display: flex;  
}  
.is-gallery {  
  flex-wrap: wrap;  
}  
.is-info {  
  flex-direction: column;  
}  
.row {  
  display: flex;  
}  
.multirow {  
  display: flex;  
  flex-wrap: wrap;  
}
```

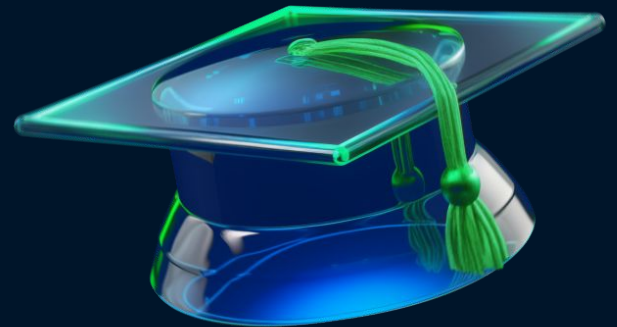
Quality Criteria for HTML Course

❤️ Use flex or grid for layouts
Do not use tables or positioning.

- ❤️ Mandatory for passing the course
- 💛 Required for the highest grade
- 💚 Optional

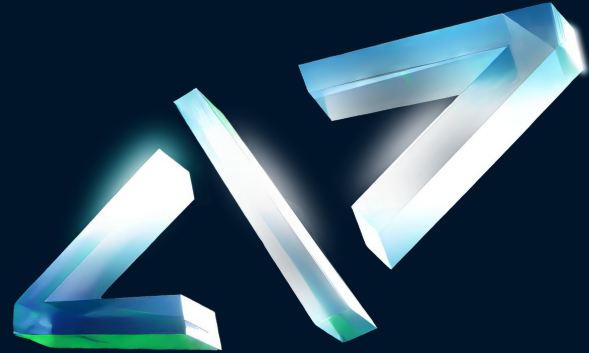
Summary

1. How to build a page layout using Flexbox



Homework

1. Create a Page Layout Using Flexbox





QUESTIONS?

Please fill out the feedback form
It's very important for us



THANK YOU!
Have a good evening!